Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- İkili arama ağacı (binary search tree, BST) tekrarı.
- BST'ye özyinelemeli (recursive) ekleme.
- BST'den silme (delete): yaprak, tek çocuklu düğüm, iki çocuklu düğüm.
- Heap ağacı (heap tree): tanımı, üç şartı, max heap ve min heap.
- Heap'in dizide saklanması.
- Heap'e ekleme ve yukarı taşıma (sift up).
- Heap sıralama (heap sort) ve aşağı taşıma (sift down), `findBigChild` fonksiyonu.

Hocanın Özellikle Vurguladığı Kısımlar

- Dengeli bir BST'de arama, ekleme ve silme $O(\log n)$ sürer. Denge bozulup ağaç tek tarafa yığılırsa $O(n)$ olur ve BST kullanmanın avantajı kalmaz.
- Ekleme her zaman uygun yaprağa yapılır. Silmede ise silinecek eleman ağacın herhangi bir yerinde olabilir, bu yüzden üç durum ayrı ele alınır.
- Hoca özyinelemeli ekleme yerine döngülü (klasik) eklemeyi tercih ettiğini söyledi. Özyinelemeli sürümün belirgin bir avantajı yok, sadece örnek olsun diye gösterildi.
- Sürekli ekleme/silme yapılan ağaçlar bağlı listede saklanmalıdır.
- Buradaki heap, bellekteki heap bölgesi (dinamik bellek ayırma) ile aynı şey değildir. Bir ağaç türüdür.
- Heap ağacı dengeli ve sola dayalı olduğu için dizide arada boş göz kalmaz. Bu yüzden heap'ler dizide saklanır.
- Heap'e ekleme yaparken hep ağacın bir kolu üzerinde ilerlenir, diğer taraf işleme girmez. Bu yüzden karmaşıklık $O(\log n)$'dir.
- Hoca BST'den silmenin kodunu yazmadı; öğrencilere bıraktı.

Kısa Tekrar Notları

- **BST'den silme:**
  - Yaprak: doğrudan silinir, ebeveyndeki pointer NULL yapılır.
  - Tek çocuk: çocuk silinen düğümün yerine geçer.
  - İki çocuk: sağ alt ağacın en küçük elemanı (inorder successor) silinen düğümün yerine konur.
- **Heap ağacının üç şartı:**
  1. Dengeli (balanced).
  2. Sola dayalı (left justified).
  3. Max heap ya da min heap özelliği.
- **Max heap:** Hiçbir düğüm ebeveyninden büyük olamaz. Kökte en büyük eleman bulunur.
- **Min heap:** Hiçbir düğüm ebeveyninden küçük olamaz. Kökte en küçük eleman bulunur.
- Heap BST değildir. Sol ve sağ çocuk arasında sıra yoktur, sadece ebeveyn-çocuk ilişkisi önemlidir.
- **Dizi indisleri (0'dan başlayarak):**
  - Sol çocuk: $2i + 1$
  - Sağ çocuk: $2i + 2$
  - Ebeveyn: $\lfloor (i - 1)/2 \rfloor$

- **Heap'e ekleme:** Eleman dizinin sonuna (n. göz) yazılır, sift up yapılır, n bir artırılır. $O(\log n)$.
- **Heapify:** Sift up ve sift down ile heap özelliğinin yeniden sağlanması.
- **Heap sort (max heap ile, artan sıra):** Kök ile son eleman yer değiştirir, bakılan bölge bir küçülür, yeni kök sift down ile yerine indirilir. Bu işlem tek eleman kalana kadar tekrarlanır.
- **Heap kullanım alanları:** heap sort ($O(n \log n)$), öncelikli kuyruklar (priority queue), bir dizinin k. büyük elemanını bulma.

Detaylı Açıklamalar

BST tekrarı ve ekleme örneği

- Kökün solundakiler kökten küçük, sağındakiler büyüktür. Bu her alt ağaç için geçerlidir.
- Örnek ağaçta (kök 22) 14 eklenirken:
  - 14 < 22, sola gidilir (11).
  - 14 > 11, sağa gidilir (13).
  - 14 > 13 ve 13 yaprak. 14, 13'ün sağına eklenir.
- 24 eklenirken: 24 > 22 sağa (25), 24 < 25 ve 25'in solu NULL. 24, 25'in soluna eklenir.

Özyinelemeli ekleme

```c
struct node {
    int value;
    struct node *left;
    struct node *right;
};

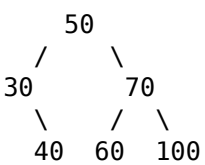
struct node *newNode(int value) {
    struct node *temp = malloc(sizeof(struct node));
    /* temp == NULL ise bellek ayrılamadı; programın ihtiyacına göre hata verilmeli */
    temp->value = value;
    temp->left = temp->right = NULL;
    return temp;
}

struct node *insert(struct node *root, int value) {
    if (root == NULL)                  /* ağaç (ya da alt ağaç) boş */
        return newNode(value);
    if (value < root->value)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);
    return root;
}
```

- Ana programda önce kök oluşturulur (ör. `root = insert(NULL, 15);`), sonra `insert(root, 17)`, `insert(root, 11)` gibi çağrılar yapılır.
- Hoca kodu adım adım izlemeyi öğrencilere bıraktı.

BST'den silme

Hocanın örnek ağacı:



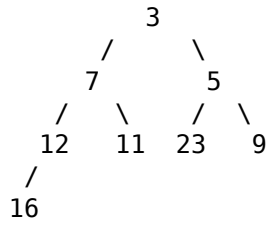
- **Durum 1, yaprak silme:** 60 silinirken 70'in sol pointer'ı NULL yapılır. En kolay durumdur.
- **Durum 2, tek çocuklu düğüm silme:** 30'un tek çocuğu 40'tır. 40, 30'un yerine geçer: 50'nin solu artık 40'ı gösterir, 30'un belleği serbest bırakılır.
  - Çocuğun solda ya da sağda olması fark etmez. Çocuk her durumda silinen düğümün ebeveynine göre aynı tarafta kalır, BST kuralı bozulmaz.
- **Durum 3, iki çocuklu düğüm silme:** 50 (kök) silinirken sağ alt ağacın (70, 60, 100) en küçük elemanı bulunur: 60.
  - 60 kökün yerine geçer. Yeni ağaç: kök 60, solda 40, sağda 70 ve 70'in sağında 100.
  - Sağ alt ağacın en küçük elemanı, sol alt ağaçtaki her şeyden büyük ve sağ alt ağaçtaki her şeyden küçüktür. Bu yüzden BST kuralı korunur.
- Hoca ikinci bir örnekte 200'ü sildi. Sağ alt ağacın en küçüğü 220 yukarı alındı.

Heap ağacı

- **Dengeli:** Seviyeler aşağıdan yukarı n, $n-1$, $n-2$, ... diye sayılırsa, 0. seviyeden $n-2$. seviyeye kadar tüm düğümlerin iki çocuğu olmalıdır.
- **Sola dayalı:** Son seviyedeki yapraklar soldan başlayarak, arada boşluk olmadan dizilir.
- **Max heap örneği:** Kök 17, çocukları 9 ve 8; 9'un çocukları 5 ve 3; 8'in çocuğu 6. Her çocuk ebeveyninden küçüktür. 9 solda, 8 sağda olmasının önemi yoktur.
- **Min heap örneği:** Kök 5, çocukları 7 ve 13; her çocuk ebeveyninden büyüktür.
- Bir tarafta ebeveynden küçük, diğer tarafta büyük değer varsa ağaç heap değildir.

Heap'in dizide saklanması

- Sıradan bir ikili ağaçta (veya BST'de) dizi kullanılırsa çok boş göz kalabilir. Ör. 2. gözün sağ çocuğu 6. gözde, onun sağ çocuğu 14. gözde olur.
- Heap'te bu sorun yoktur. Örnek min heap:



- Dizi hali: `3, 7, 5, 12, 11, 23, 9, 16`.

Heap'e ekleme (min heap)

- Yeni eleman dizinin ilk boş gözüne (son göz) yazılır.
- Ebeveyni ile karşılaştırılır. Ebeveyn daha büyükse yer değiştirirler.
- Bu işlem eleman köke ulaşana ya da ebeveyni kendinden küçük olana kadar sürer. Bu yukarı taşıma işlemine sift up denir.
- Adım adım örnek:

Gelen	İşlem	Dizi (sonuç)
20	ilk eleman	20
15	15 < 20, yer değiştir	15, 20
23	23 > 15, yerinde kalır	15, 20, 23
8	8 < 20 değiştir, 8 < 15 değiştir	8, 15, 23, 20
12	12 < 15 değiştir, 12 > 8 dur	8, 12, 23, 20, 15
11	11 < 23 değiştir, 11 > 8 dur	8, 12, 11, 20, 15, 23

- Son boş gözü bulmak için eleman sayısı n tutulur. Yeni eleman n. göze yazılır, sonra n bir artırılır.
- Ebeveyn indisi örneği: Dizi `4, 7, 5, 12, 21, 13, 20` ve 10 ekleniyor.
  - 10, 7. göze yazılır. Ebeveyni $(7-1)/2 = 3$. gözdeki 12'dir. 10 < 12, yer değiştirir.
  - Yeni yeri 3. göz. Ebeveyni $(3-1)/2 = 1$. gözdeki 7'dir. 10 > 7, durur.

  - Sonuç: `4, 7, 5, 10, 21, 13, 20, 12`.

```
void insert(int item, int heap[], int *n) {
    heap[*n] = item;
    siftUp(item, heap, *n);
    (*n)++;
}

void siftUp(int item, int heap[], int address) {
    int parent, stop = 0;
    while (address > 0 && !stop) {
        parent = (address - 1) / 2;
        if (heap[parent] > item) {          /* min heap */
            swap(&heap[address], &heap[parent]);
            address = parent;
        } else {
            stop = 1;
        }
    }
}
```

- İzleme örneği: Dizi `5, 11, 7, 13, 12, 16` ($n = 6$), 3 ekleniyor.
  - 3, 6. göze yazılır. Ebeveyn 2. göz (7). 7 > 3, yer değiştir, adres 2.
  - Ebeveyn 0. göz (5). 5 > 3, yer değiştir, adres 0.
  - Adres 0, döngü biter. Sonuç: `3, 11, 5, 13, 12, 16, 7`.

Heap sort

- Max heap'te kök her zaman en büyük elemandır. Bu özellik sıralamada kullanılır.
- Örnek max heap: `22, 15, 17, 14, 11, 9`.
  - 22 ile son eleman 9 yer değiştirir. 22 artık doğru yerindedir ve bir daha bakılmaz.
  - 9 kökte heap özelliğini bozar. Önce iki çocuk (15 ve 17) karşılaştırılır, büyük olan (17) 9 ile karşılaştırılır. 17 > 9, yer değiştirir.
  - Sonuç: `17, 15, 9, 14, 11 | 22`.
  - İkinci adım: 17 ile 11 yer değiştirir (`11, 15, 9, 14 | 17, 22`). 11'in çocukları 15 ve 9; büyük olan 15 yukarı çıkar. 11'in yeni yerindeki çocuğu 14 de 11'den büyük olduğu için bir kez daha yer değiştirilir: `15, 14, 9, 11 | 17, 22` (bkz. ders sonrası notlar).
  - Bu işlem tekrarlanır. Sonunda dizi `9, 11, 14, 15, 17, 22` olur.
- Neden önce iki çocuk karşılaştırılır? Küçük çocuk yukarı alınırsa bir adım sonra yine düzeltme gerekir. Büyük çocuk yukarı alınırsa heap özelliği tek adımda sağlanır.

```
void heapSort(int heap[], int n) {
    for (int i = n - 1; i > 0; i--) {
        swap(&heap[0], &heap[i]);   /* en büyük eleman sona */
        siftDown(heap, i);          /* yalnız ilk i elemana bakılır */
    }
}

void siftDown(int heap[], int n) {
    int parent = 0;
    int child = findBigChild(heap, parent, n);
    while (child != -1 && heap[child] > heap[parent]) {
        swap(&heap[child], &heap[parent]);
        parent = child;
        child = findBigChild(heap, parent, n);
    }
}
```

`findBigChild` fonksiyonu

Üç durum vardır:

1. **İki çocuk var** ($n > 2p + 2$): Büyük olan çocuğun indisi döndürülür.
2. **Tek çocuk var** ($2p + 1 < n$ ama $2p + 2 \geq n$): Sol çocuğun indisi ($2p + 1$) döndürülür.
3. **Çocuk yok** (ebeveyn yaprak): -1 döndürülür. `siftDown` döngüsü durur.

```
int findBigChild(int heap[], int parent, int n) {
    if (n > 2 * parent + 2) {                       /* iki çocuk */
        if (heap[2 * parent + 1] > heap[2 * parent + 2])
            return 2 * parent + 1;
        else
            return 2 * parent + 2;
    } else if (2 * parent + 1 < n) {                /* tek çocuk */
        return 2 * parent + 1;
    } else {
        return -1;                                  /* yaprak */
    }
}
```

- İzleme örneği: Dizi `15, 8, 12, 4, 7`.

Adım	İşlem	Dizi
$i = 4$	15 ile 7 yer değiştir; 7 ile büyük çocuk 12 yer değiştir	12, 8, 7, 4, 15
$i = 3$	12 ile 4 yer değiştir; 4 ile büyük çocuk 8 yer değiştir	8, 4, 7, 12, 15
$i = 2$	8 ile 7 yer değiştir; 7'nin tek çocuğu 4 küçük, dur	7, 4, 8, 12, 15
$i = 1$	7 ile 4 yer değiştir	4, 7, 8, 12, 15

Ders sonrası notlar

- **Sağ alt ağacın en küçüğü her zaman yaprak değildir.** Hoca "en küçük eleman yaprakta olacağı için" dedi. Doğrusu: Bu elemanın sol çocuğu yoktur ama sağ çocuğu olabilir. Sağ çocuğu varsa, en küçük eleman yukarı alındıktan sonra onun sağ çocuğu eski yerine bağlanır (tek çocuklu silme durumu). Sol alt ağacın en büyük elemanı (inorder predecessor) da aynı amaçla kullanılabilir.
- **Min heap örneğindeki hata:** Hoca min heap örneğinde 13'ün altına 11 yazdı. Bir öğrenci 11 < 13 olduğu için heap özelliğinin bozulduğunu fark etti. Değer 14 yapılarak düzeltildi.
- **`siftUp` kodunda eksik:** Hoca `parent` değişkenini tanımlamayı unuttu. Bir öğrenci uyardı ve düzeltildi.
- **Max heap sıralama örneği:** Hoca örnekte önce 17 yerine yanlışlıkla 7 yazdı, öğrenci uyarınca düzeltildi.
- **Dengeli olmayan örnek ağaç:** Bir örnekte 5. gözdeki düğüm sola dayalı olmayan bir yerdeydi. Bir öğrenci $2i + 1$ kuralına göre yerinin farklı olması gerektiğini fark etti. Hoca kabul edip ağacı düzeltti.
- **Heap sort'un başlangıcı:** Hocanın örnekleri hazır bir heap ile başladı. Sıralanmamış bir diziyle başlanırsa önce diziden heap oluşturulmalıdır (build heap). Bu, ya elemanları tek tek ekleyerek ($O(n \log n)$) ya da son iç düğümden köke doğru her düğüme sift down uygulayarak ($O(n)$) yapılır. Heap sort'un toplam karmaşıklığı $O(n \log n)$'dir.
- **Heap sort örneğinin ikinci adımı:** Hoca ikinci adımda 15 ile 11'i değiştirip `15, 11, 9, 14 | 17, 22` sonucunda durdu. Bu bir max heap değildir: 1. gözdeki 11'in çocuğu (3. göz) 14'tür ve 14 > 11. Sift down sürdürülmelidir; doğru sonuç `15, 14, 9, 11 | 17, 22`'dir (python ile denendi).
- **`siftUp`'ta `address > 0`:** Hoca adresin 0 olmasını "ağaç boş" diye yorumladı. Doğrusu: Adres 0, elemanın köke ulaştığı anlamına gelir; kökün ebeveyni olmadığı için yukarı taşıma biter.
- **Ekleme karmaşıklığı örneği:** Hoca "8 elemanınız varsa 3 seviyeniz var" dedi. 8 elemanlı heap'te 4 seviye (0-3) vardır; yükseklik 3'tür, bu yüzden sift up en çok 3 adım sürer.
- **Terim:** Altyazıda "shift up/shift down" geçer. Literatürdeki terim sift up / sift down'dur.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.